Transformers

- Context Matters

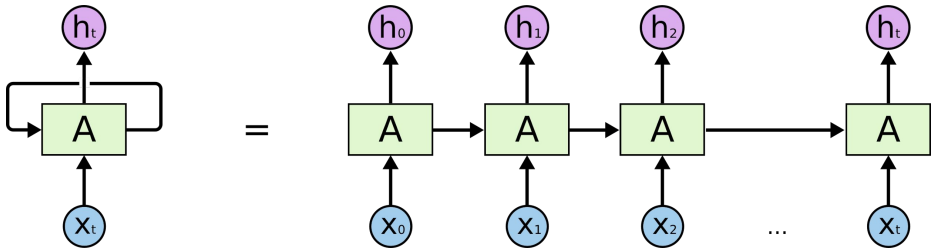
Luís Filipe Cunha
lfc@di.uminho.pt

José João Almeida
jj@di.uminho.pt

Before Transformers

Before Transformers

RNN sequential processing - slow
Vanishing gradient - information loss
CNN - limited context (fixed window)
LSTM and **Bi-LSTM** to the rescue!

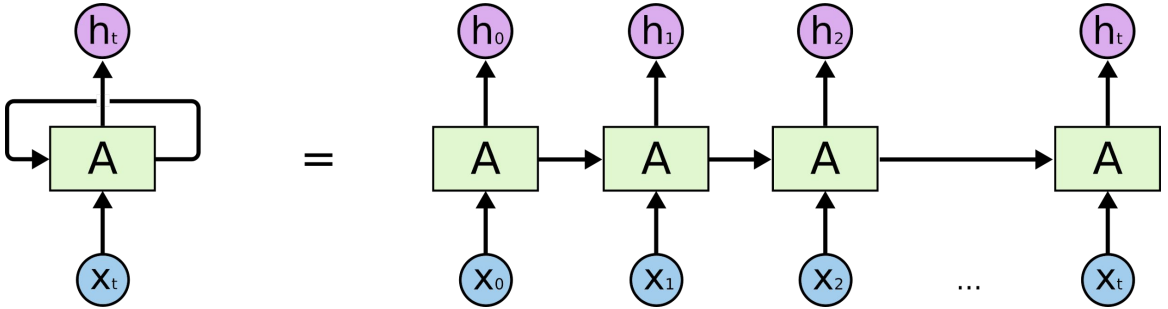


- Complex;
- Difficult to train;
- Sequential ->The model can only reason about a word when all the previous words are already processed

RNN

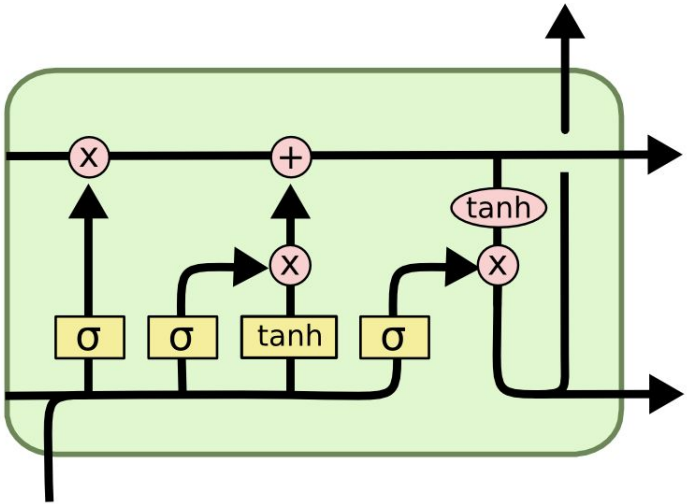
Problems:

- Vanishing Gradient
- Long term Dependencies
- Only one direction



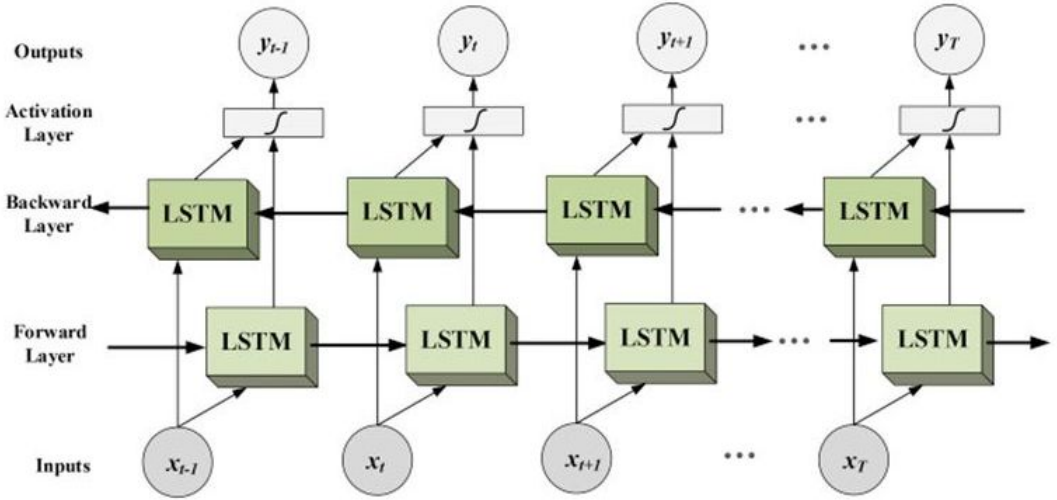
Long Short Term Memory

- RNN with Long Term Memory
- Introduction of a Memory Cell
- Input Gate
- Output Gate
- Forget Gate



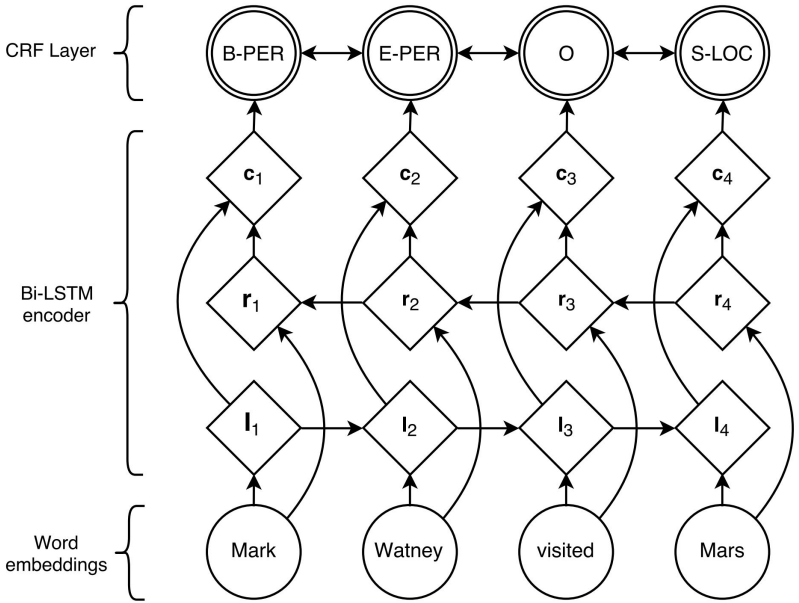
Bidirectional-LSTM

- Take in account the whole document
- More computational Resources



BiLSTM-CRF

- Sentence-level tag information
- Past and future tags
- Decoding the best tagging sequence, increasing the tagging accuracy



Relevant Works

Computer Science > Computation and Language

[Submitted on 15 Feb 2018 (v1), last revised 22 Mar 2018 (this version, v2)]

Deep contextualized word representations

Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, Luke Zettlemoyer

We introduce a new type of deep contextualized word representation that models both (1) complex characteristics of word use (e.g., syntax and semantics), and (2) how these uses vary across linguistic contexts (i.e., to model polysemy). Our word vectors are learned functions of the internal states of a deep bidirectional language model (biLM), which is pre-trained on a large text corpus. We show that these representations can be easily added to existing models and significantly improve the state of the art across six challenging NLP problems, including question answering, textual entailment and sentiment analysis. We also present an analysis showing that exposing the deep internals of the pre-trained network is crucial, allowing downstream models to mix different types of semi-supervision signals.

Comments: NAACL 2018. Originally posted to openreview 27 Oct 2017. v2 updated for NAACL camera ready
Subjects: **Computation and Language (cs.CL)**
Cite as: arXiv:1802.05365 **[cs.CL]**
(or arXiv:1802.05365v2 **[cs.CL]** for this version)

Submission history

From: Matthew Peters [view email]
[v1] Thu, 15 Feb 2018 00:05:11 UTC (135 KB)
[v2] Thu, 22 Mar 2018 21:59:40 UTC (140 KB)

Computer Science > Computation and Language

[Submitted on 18 Jan 2018 (v1), last revised 23 May 2018 (this version, v5)]

Universal Language Model Fine-tuning for Text Classification

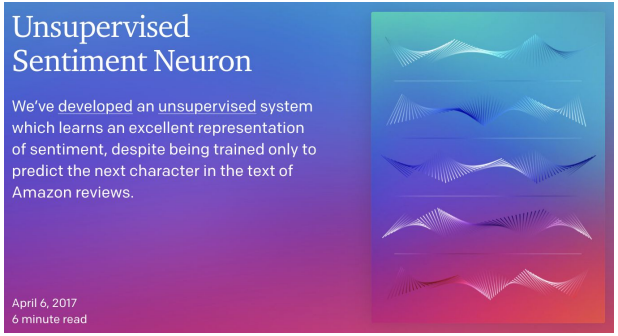
Jeremy Howard, Sebastian Ruder

Inductive transfer learning has greatly impacted computer vision, but existing approaches in NLP still require task-specific modifications and training from scratch. We propose Universal Language Model Fine-tuning (ULMFiT), an effective transfer learning method that can be applied to any task in NLP, and introduce techniques that are key for fine-tuning a language model. Our method significantly outperforms the state-of-the-art on six text classification tasks, reducing the error by 18-24% on the majority of datasets. Furthermore, with only 100 labeled examples, it matches the performance of training from scratch on 100x more data. We open-source our pretrained models and code.

Comments: ACL 2018, fixed denominator in Equation 3, line 3
Subjects: **Computation and Language (cs.CL)**; Machine Learning (cs.LG); Machine Learning (stat.ML)
Cite as: arXiv:1801.06146 **[cs.CL]**
(or arXiv:1801.06146v5 **[cs.CL]** for this version)

Submission history

From: Sebastian Ruder [view email]
[v1] Thu, 18 Jan 2018 17:54:52 UTC (30 KB)
[v2] Mon, 14 May 2018 13:57:04 UTC (851 KB)



NATURAL LANGUAGE PROCESSING

NLP's ImageNet moment has arrived

Big changes are underway in the world of NLP. The long reign of word vectors as NLP's core representation technique has seen an exciting new line of challengers emerge. These approaches demonstrated that pretrained language models can achieve state-of-the-art results and herald a watershed moment.

SEBASTIAN RUDER
12 JUL 2018 · 16 MIN READ

Transformers

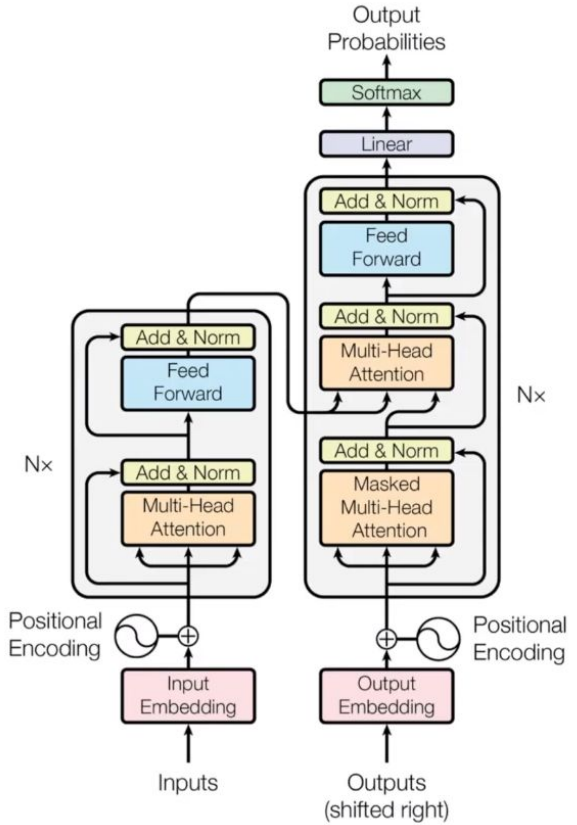
Attention Is All You Need

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks in an encoder-decoder configuration. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

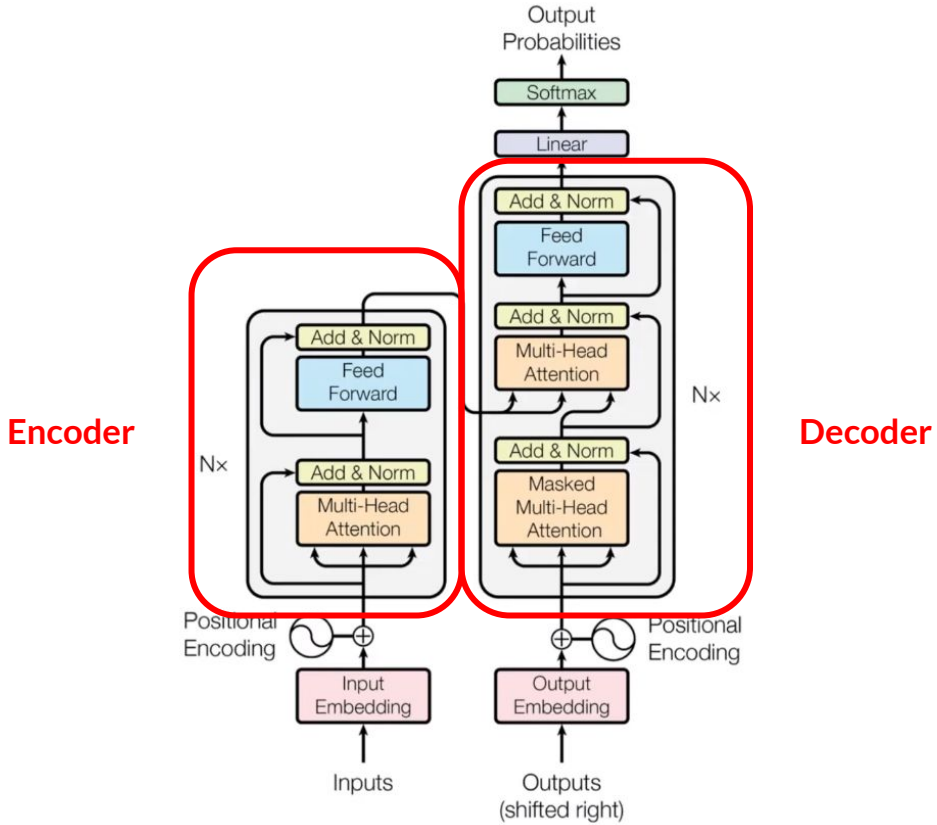
What is a Transformer?

- Introduced in the paper *"Attention is All You Need"* (Vaswani et al., 2017)
- Encoder - Decoder architecture
- **Attention Mechanism**
- **Positional Encodings**
- **Easier to parallelize (unlike LSTMs)**

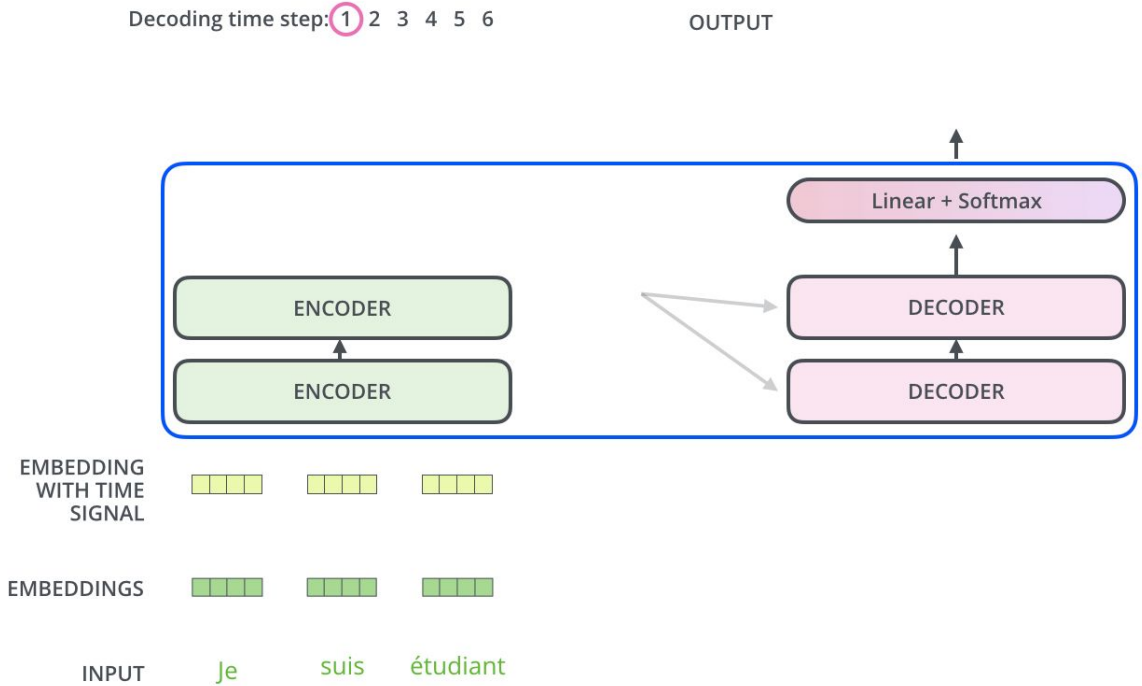


Encoder Decoder

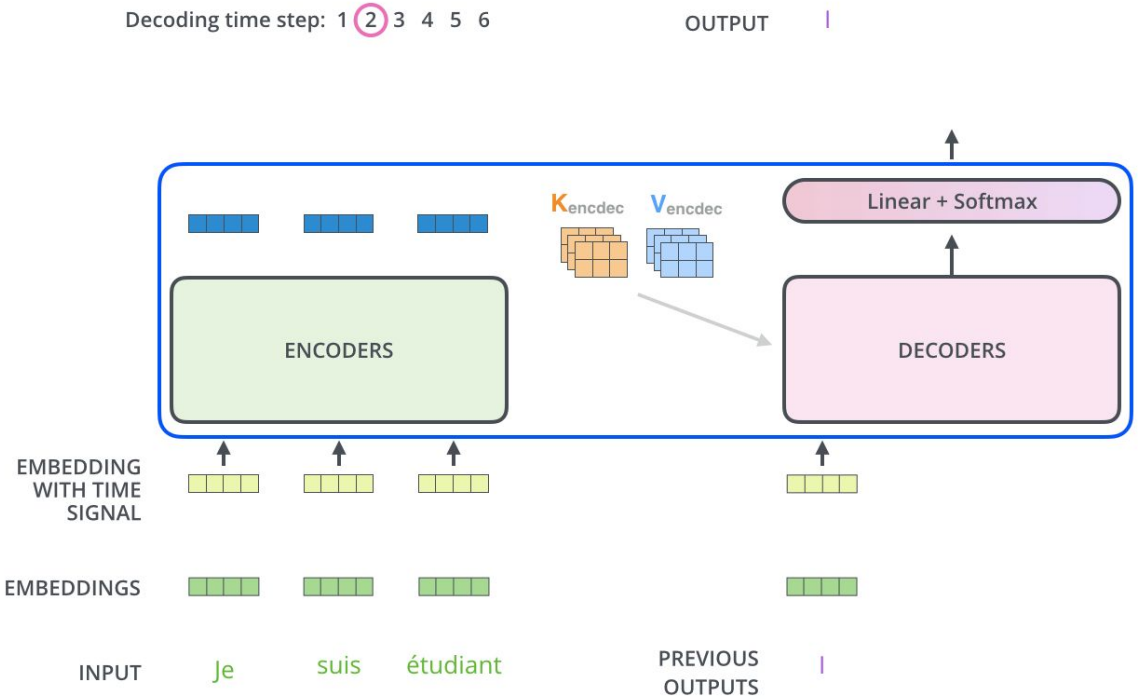
- Originally developed for seq-to-seq tasks (translation, summarization, …)
- **Encoder**: Processes the input data and outputs a sequence of vectors.
- **Decoder**: Takes the encoder's output and generates a sequence of predictions.



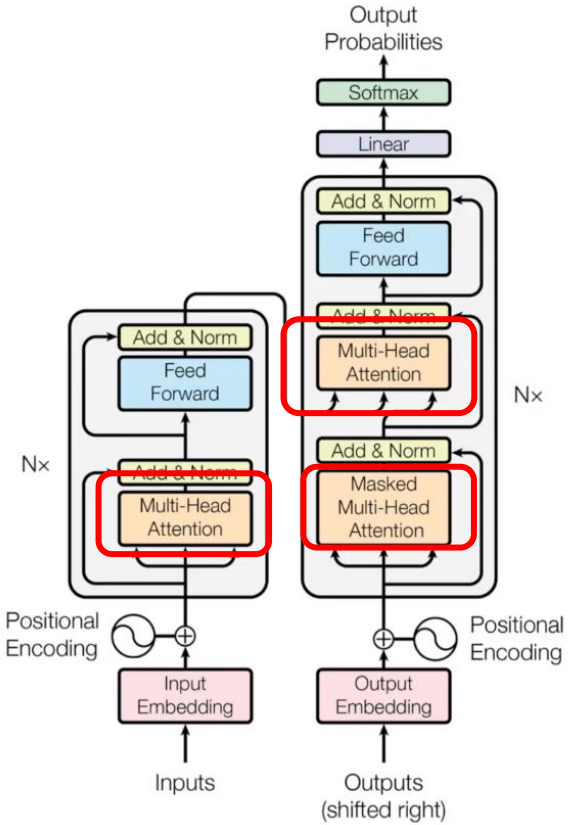
Encoder


Decoding time step: 1 2 3 4 5 6
OUTPUT
Linear + Softmax
ENCODER
ENCODER
DECODER
DECODER
EMBEDDING WITH TIME SIGNAL
EMBEDDINGS
INPUT
Je
suis
étudiant

Decoder


Decoding time step: 1 2 3 4 5 6
OUTPUT
I
Linear + Softmax
Kencdec
Vencdec
ENCODERS
DECODERS
EMBEDDING WITH TIME SIGNAL
EMBEDDINGS
INPUT
Je
suis
étudiant
PREVIOUS OUTPUTS
I

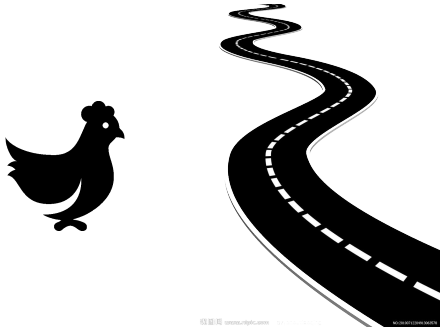
Self Attention



Self-Attention

Consider the following example:

- The chicken didn't cross the road because ***it*** was too tired.
- The chicken didn't cross the road because ***it*** was too wide.



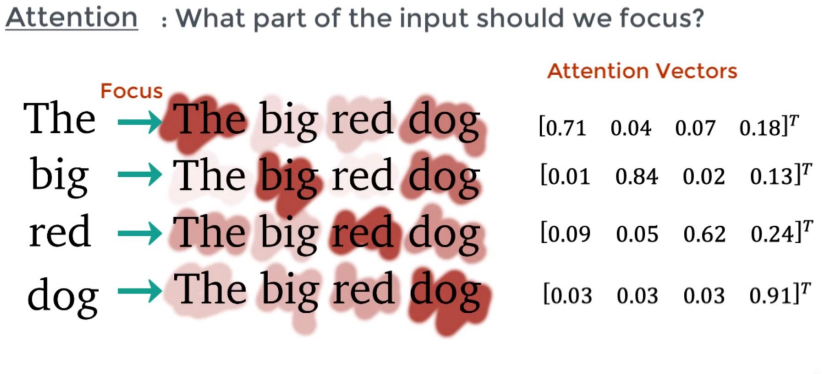
If we are to compute the meaning of this sentence, we'll need the meaning of ***it*** to be associated with the chicken in the first sentence and associated with the road in the second one

Contextual words that help us compute the meaning of words in context can be quite far away in the sentence or paragraph.

Self-Attention

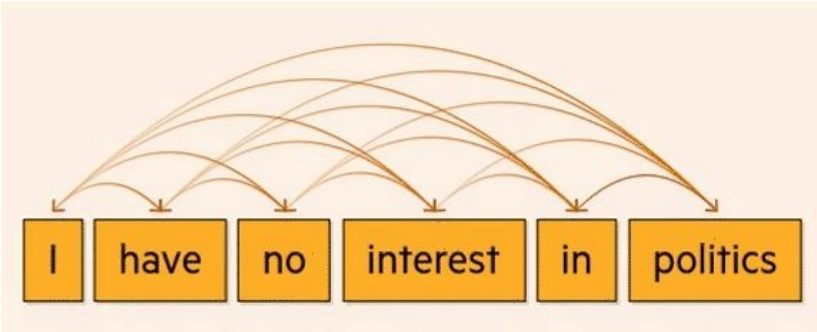
- **Self-Attention** allows each word in the input sequence to focus on different parts of the sequence.

I have no interest in politics



Self-attention looks at each token in a body of text and decides which others are most important to understand its meaning.

Self-Attention



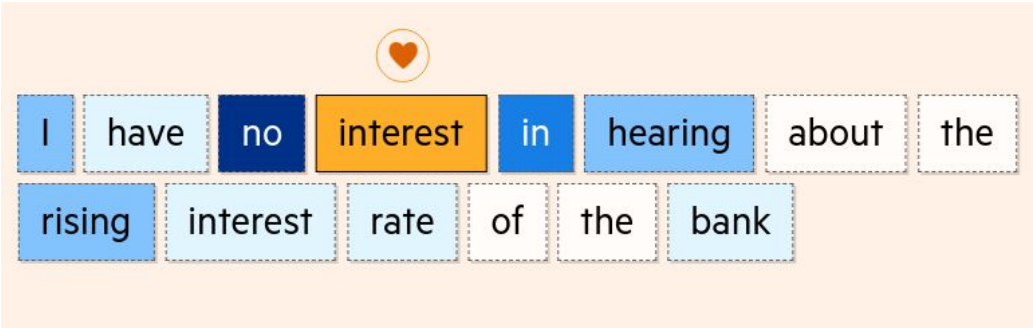
Assessing the whole sentence at once means the transformer is able to understand that interest is being used as a noun to explain an individual's take on politics.

If we tweak the sentence, the model understands interest is now being used in a financial sense.

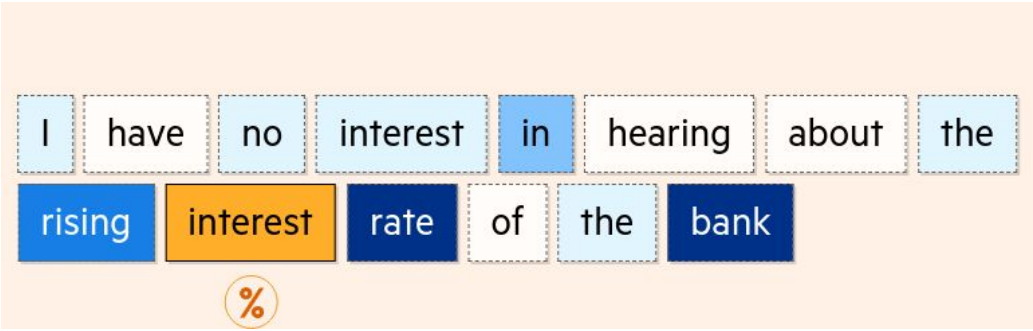
Self-Attention

When we combine the sentences, the model is still able to recognise the correct meaning of each word thanks to the attention it gives the accompanying text.

For the first use of interest, it is no and in that are most attended.

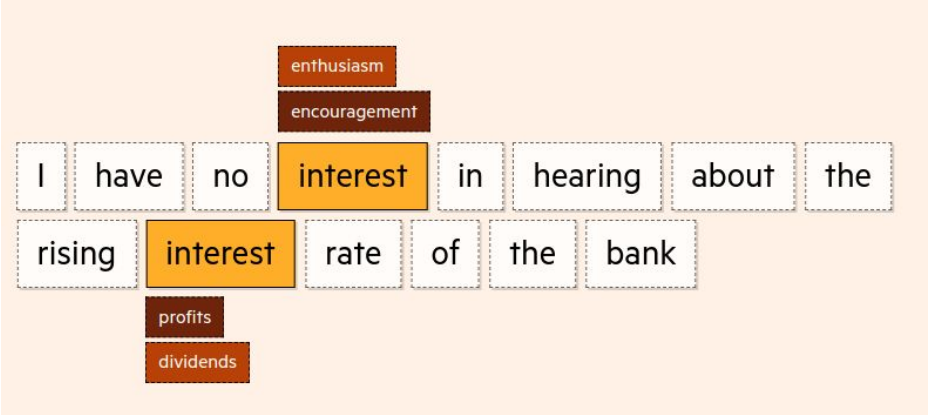


For the second, it is rate and bank.

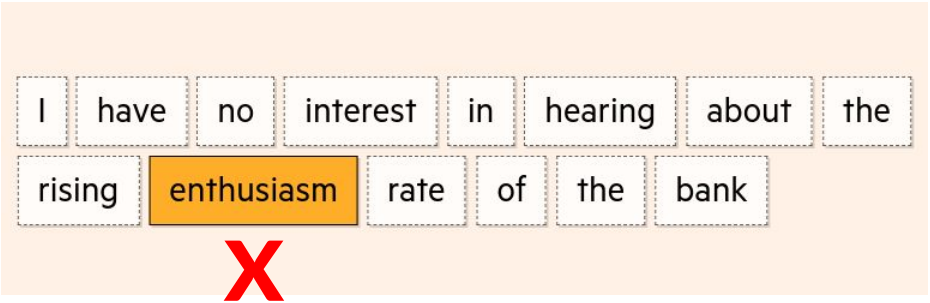


Self-Attention

This functionality is crucial for advanced text generation. Without it, words that can be interchangeable in some contexts but not others can be used incorrectly.



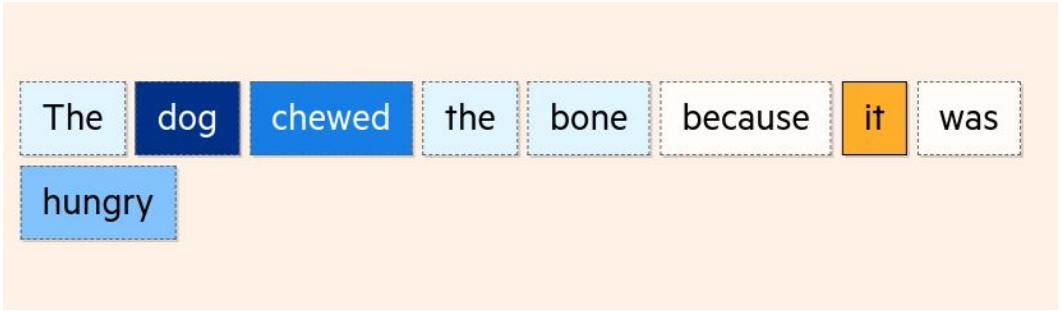
Effectively, self-attention means that if a summary of this sentence was produced, you wouldn't have enthusiasm used when you were writing about interest rates.



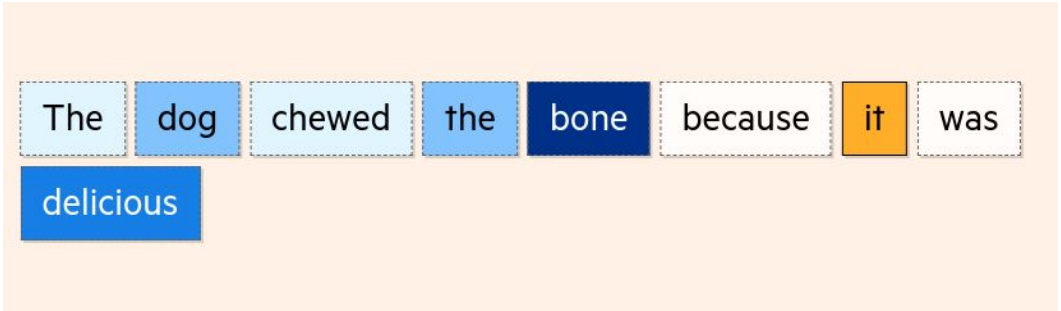
Self-Attention

This capability goes beyond words, like interest, that have multiple meanings.

Self-attention is able to calculate that it is most likely to be referring to dog.

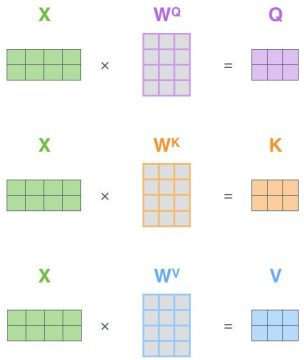


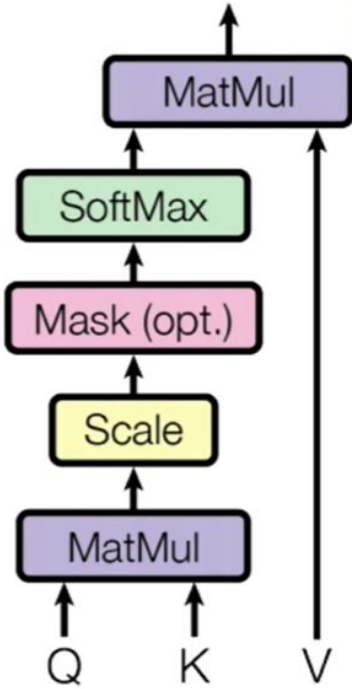
And if we alter the sentence, swapping hungry for delicious, the model is able to recalculate, with it now most likely to refer to bone.



Self Attention

- Each token in the input sequence attend to every other word in the sequence;
- Richer representation of the word by considering the entire context.
- For each word we generate three vectors:
  - **Query (Q)**: Represents the current word.
  - **Key (K)**: Represents the words to which the current word will attend.
  - **Value (V)**: Represents the actual content that each word contributes to the current word's output. Once a word is deemed relevant (via the Q-K match) the Values of those words are combined



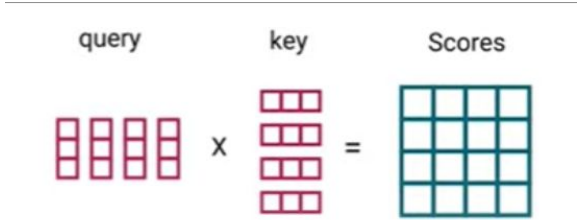


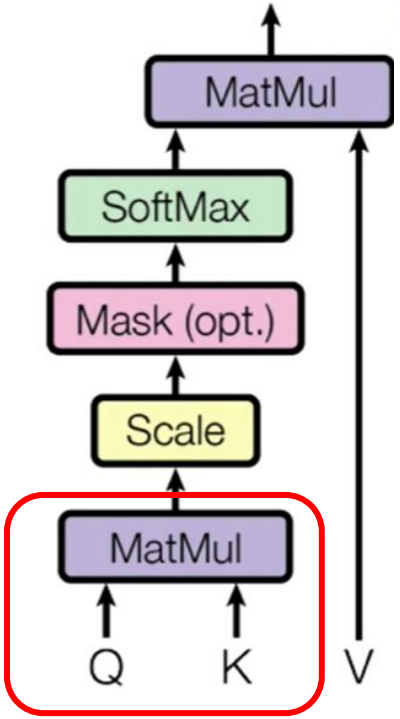
Attention scores

The **attention score** for a given word (query) attending to another word (key) is computed as the **dot product** of the query and key vectors.

This score determines how much focus (weight) the model should give to each word when computing the output.

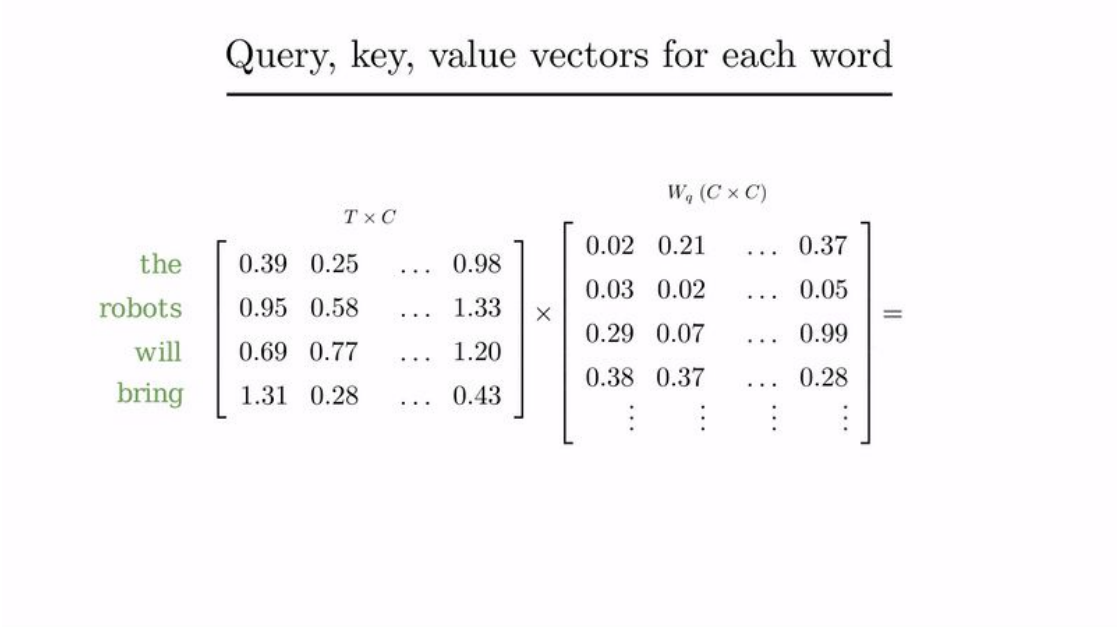
$$\text{Attention Score} = Q \cdot K^T$$





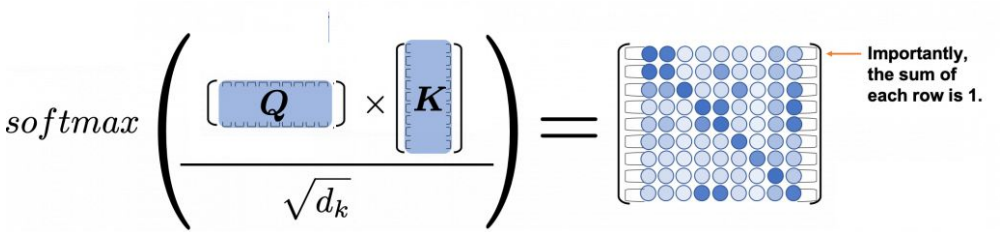
Attention Score

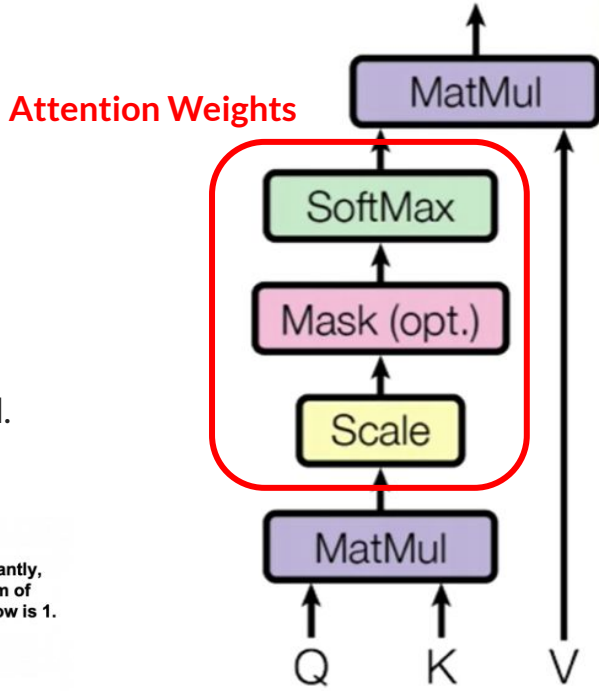
Query, Key, Value vectors



Attention Weights

- Scaled down the scores - more stable gradients to avoid exploding effects of multiplication.
- Softmax to get the **attention weights** (probability [0, 1])
- Higher scores get enhanced, and lower scores are depressed.
- Model learns which words to attend to.

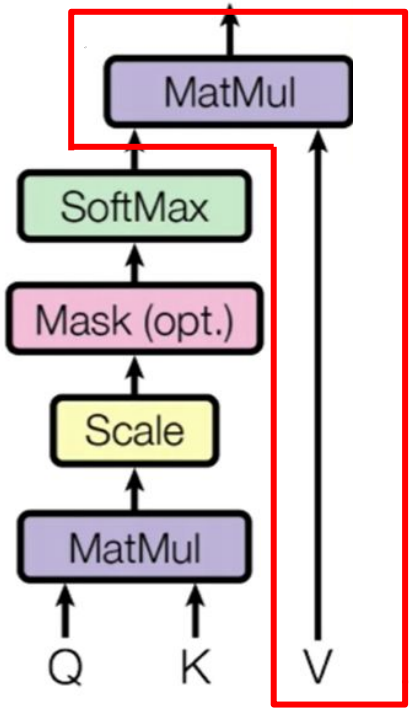




Self Attention

- **Attention weights** * **Value (V)**
- The higher softmax scores will keep the value of words the model learns is **more important**. The lower scores will drown out the **irrelevant** words.

$$\text{Attention}(Q, K, V) = \texttt{softmax}\left(\frac{Q \times K^T}{\sqrt{d_k}}\right) V$$



Attention weights * Value (V)

ORIGINAL VIEW

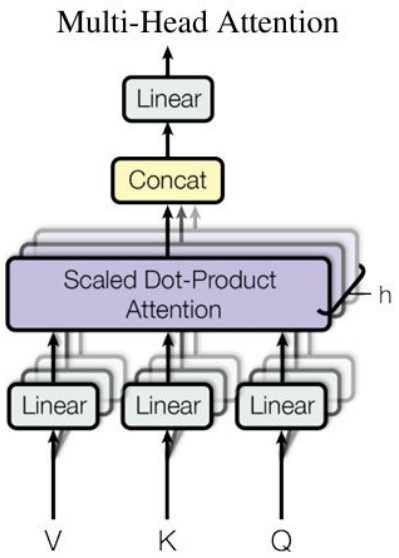
FOCUSED VIEW

Attention weights * Value (V)

Multi-Head Attention

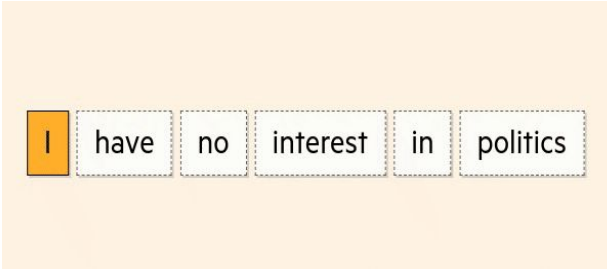
- Multiple sets of Query/Key/Value weight matrices (8 by default)
- Each set is randomly initialized (during training).
- Then, after training, each set is used to project the input embeddings into a different representation subspace.

Multiple heads allow the model to **look at different parts of the sentence** from **different perspectives**

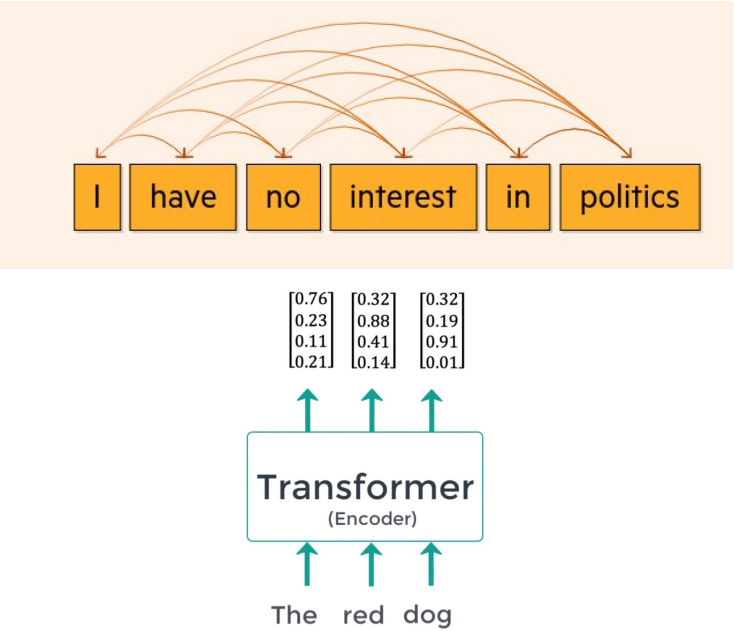


Parallelization

RNNs

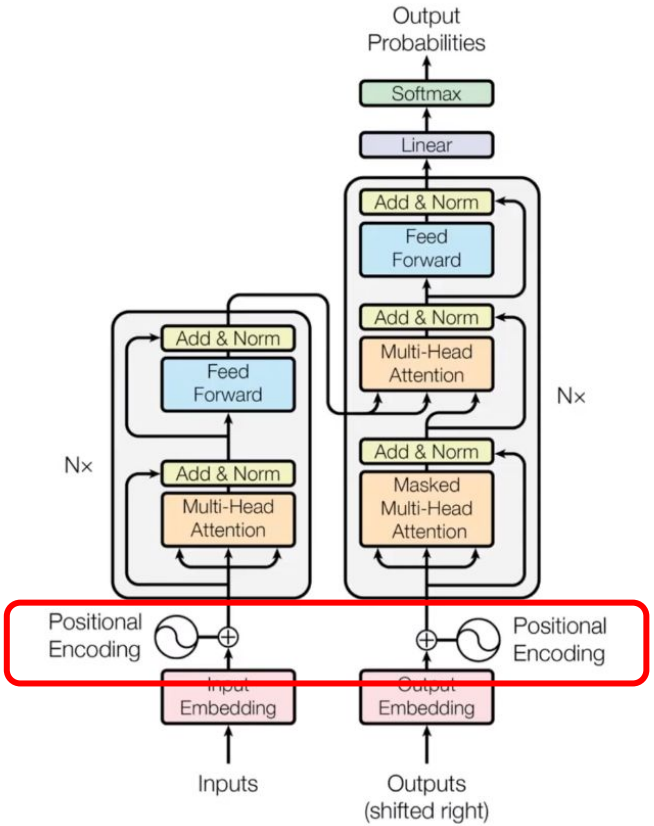


Transformer



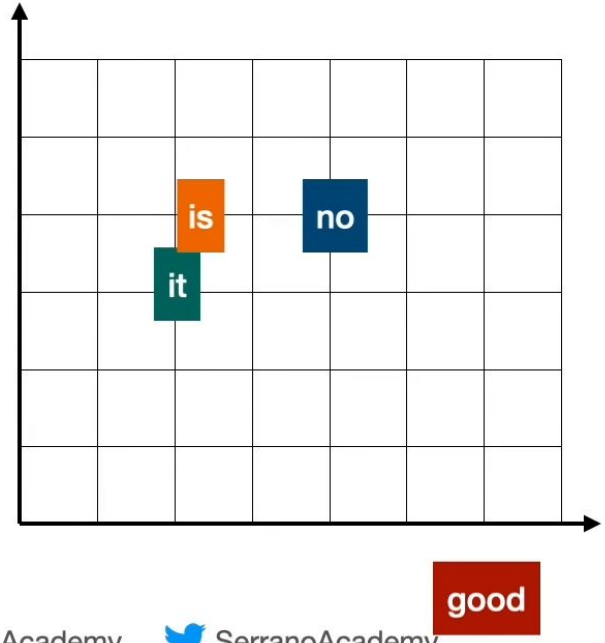
- Unlike **RNNs** or **LSTMs**, which process inputs sequentially, the **Transformer** processes all words in a sentence simultaneously.

Positional Encoding

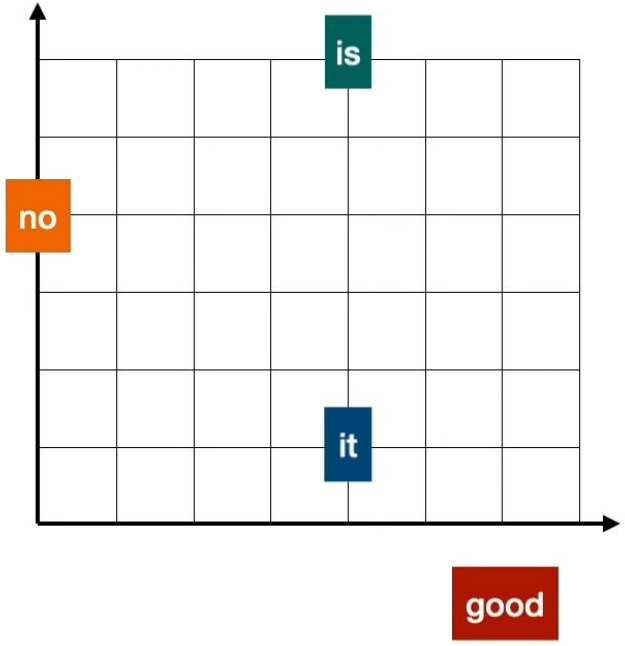


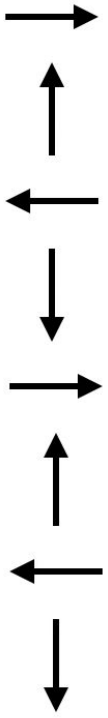
Positional encoding

Repeats after the 4th word



different!





Positional Encodings

“We only need to have a consistent way of changing the words”

Key Ideas:

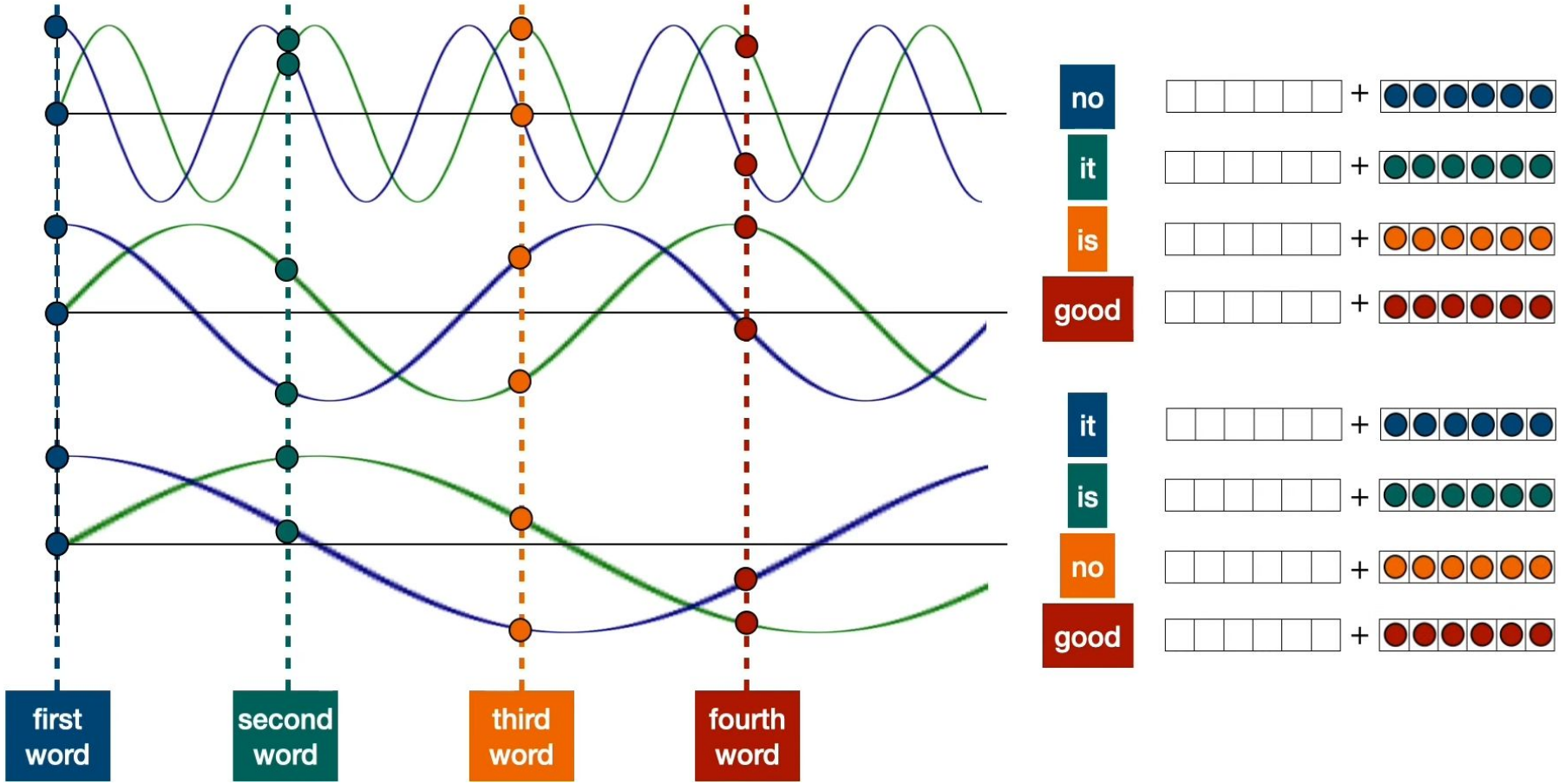
- Each position gets a **unique**, consistent vector.
- Small shifts in original vectors ensure semantic stability.
- The encoding method must be simple and easy to learn

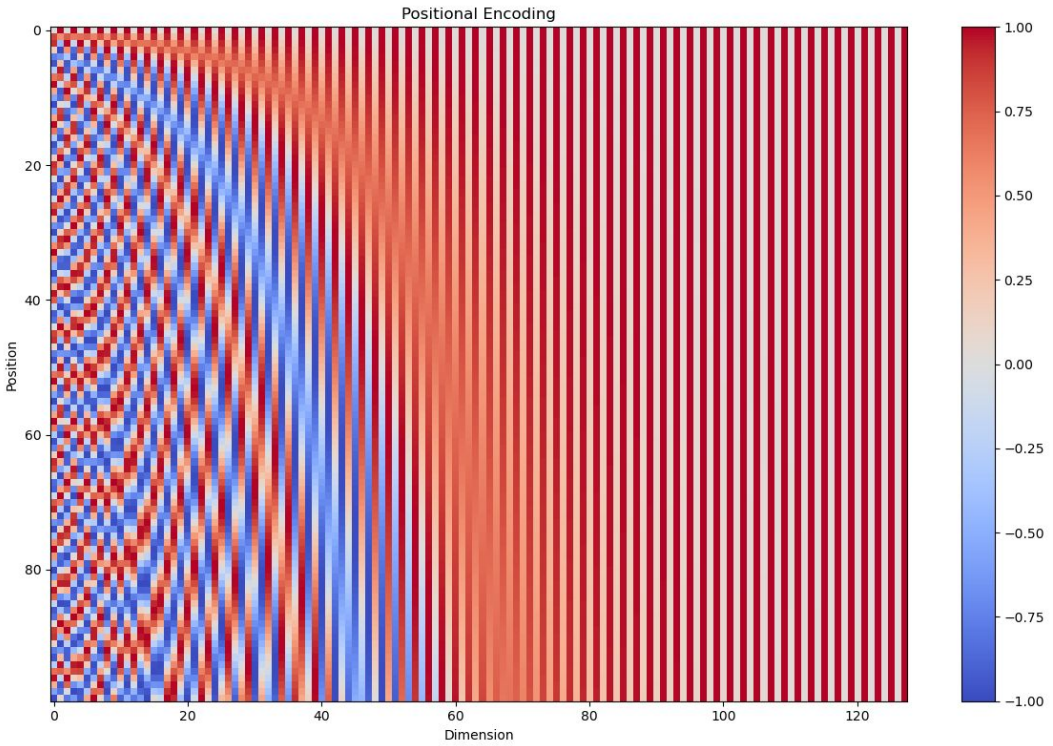
Note: Neural networks are **spurious correlation identifying beasts**.
The model sees the pattern so often during training that it learns to decode it.

Positional Encoding

$$PE_{(pos,2i)} = sin(pos/10000^{2i/d_{\text{model}}})$$
$$PE_{(pos,2i+1)} = cos(pos/10000^{2i/d_{\text{model}}})$$

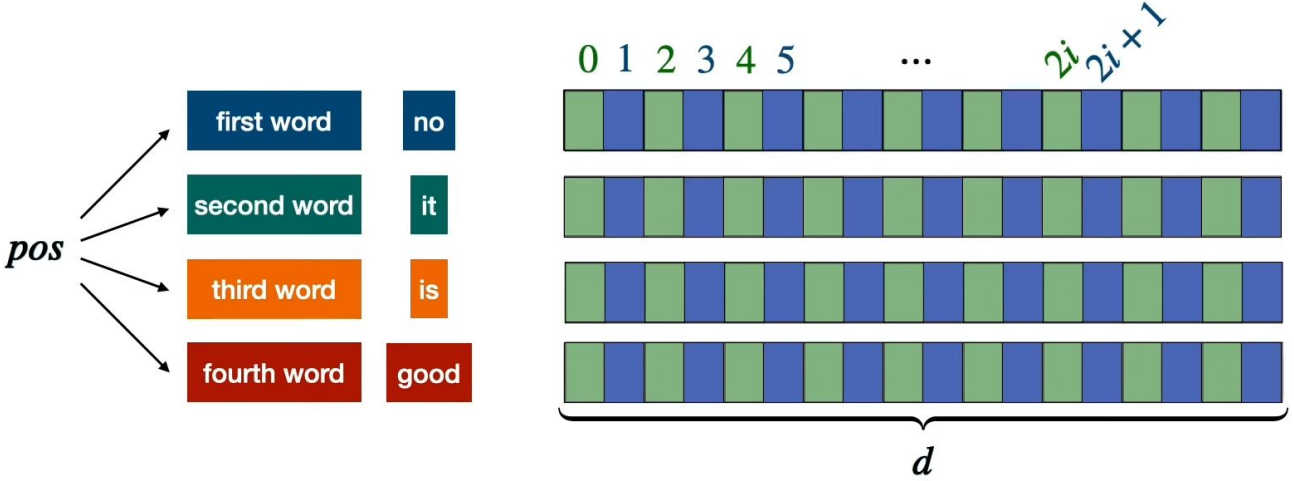
- sine and cosine functions (sinusoidal functions) to generate the positional encodings
  - `pos`: token position (0, 1, 2, ...)
  - `i`: dimension index
  - d: embedding dimension


first word
second word
third word
fourth word
no
it
is
good
it
is
no
good


Positional Encoding
Position
Dimension
0
20
40
60
80
100
120
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00

Positional Encoding

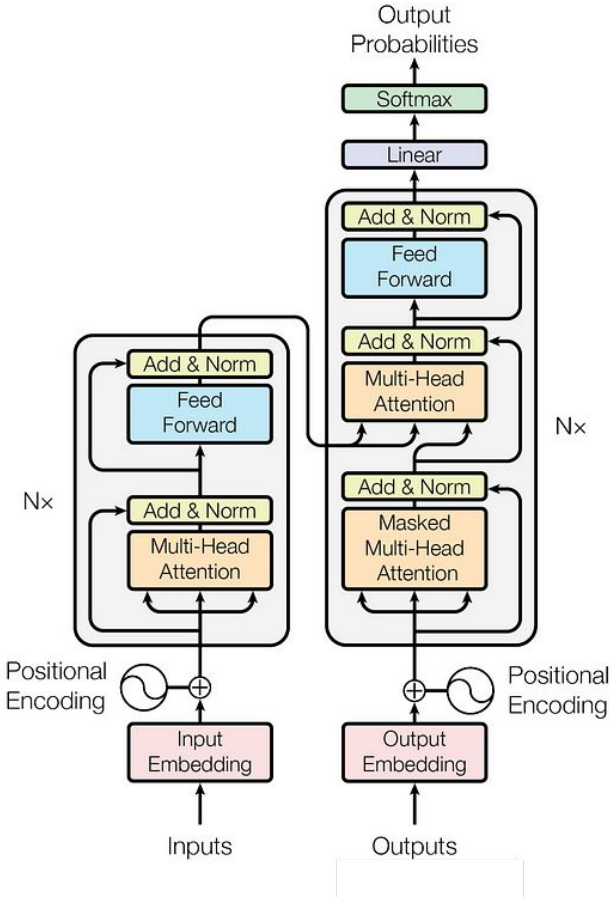
$$PE(pos,2i) = \cos\left(\frac{pos}{10000^{2i/d}}\right) \qquad PE(pos,2i+1) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$



Transformer Models

BERT

Encoder



GPT

Decoder

Transformer Models

Computer Science > Computation and Language

[Submitted on 11 Oct 2018 (v1), last revised 24 May 2019 (this version, v2)]

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova

We introduce a new language representation model called BERT, which stands for Bidirectional Encoder Representations from Transformers. Unlike recent language representation models, BERT is designed to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers. As a result, the pre-trained BERT model can be fine-tuned with just one additional output layer to create state-of-the-art models for a wide range of tasks, such as question answering and language inference, without substantial task-specific architecture modifications.

BERT is conceptually simple and empirically powerful. It obtains new state-of-the-art results on eleven natural language processing tasks, including pushing the GLUE score to 80.5% (7.7% point absolute improvement), MultiNLI accuracy to 86.7% (4.6% absolute improvement), SQuAD v1.1 question answering Test F1 to 93.2 (1.5 point absolute improvement) and SQuAD v2.0 Test F1 to 83.1 (5.1 point absolute improvement).

Subjects: **Computation and Language (cs.CL)**
Cite as: arXiv:1810.04805 **[cs.CL]**
(or arXiv:1810.04805v2 **[cs.CL]** for this version)

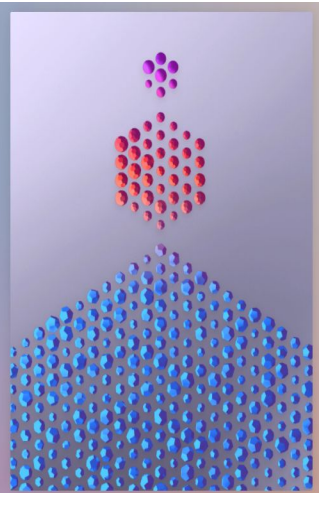
Submission history

From: Ming-Wei Chang [view email]
[v1] Thu, 11 Oct 2018 00:50:01 UTC (227 KB)
[v2] Fri, 24 May 2019 20:37:26 UTC (309 KB)

Improving Language Understanding with Unsupervised Learning

We've obtained state-of-the-art results on a suite of diverse language tasks with a scalable, task-agnostic system, which we're also releasing. Our approach is a combination of two existing ideas: transformers and unsupervised pre-training. These results provide a convincing example that pairing supervised learning methods with unsupervised pre-training works very well; this is an idea that many have explored in the past, and we hope our result motivates further research into applying this idea on larger and more diverse datasets.

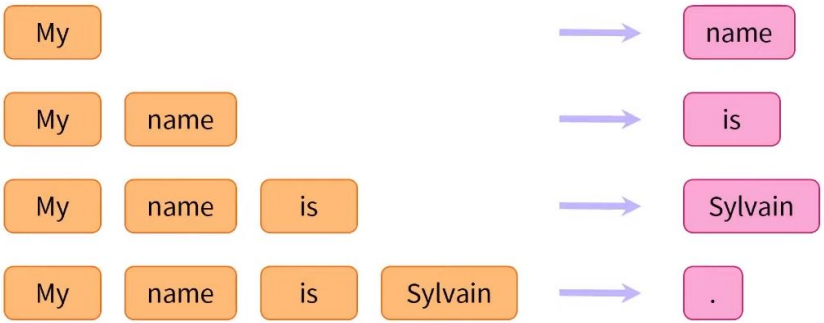
June 11, 2018
9 minute read



Pre-Training tasks

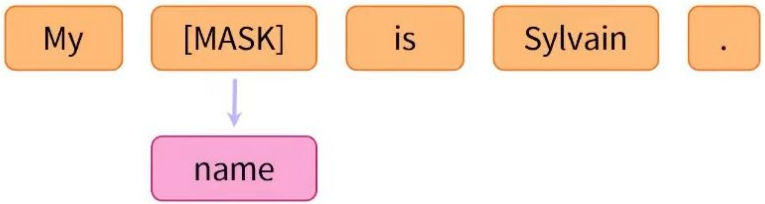
Auto-Regressive Language Modeling - GPT

- Predict the **next word** given all previous words.
- It only looks **to the left** — past tokens.



Masked Language Modeling (MLM) - BERT

- Randomly mask out some tokens from the input.
- The model must **predict the masked tokens** based on the context.

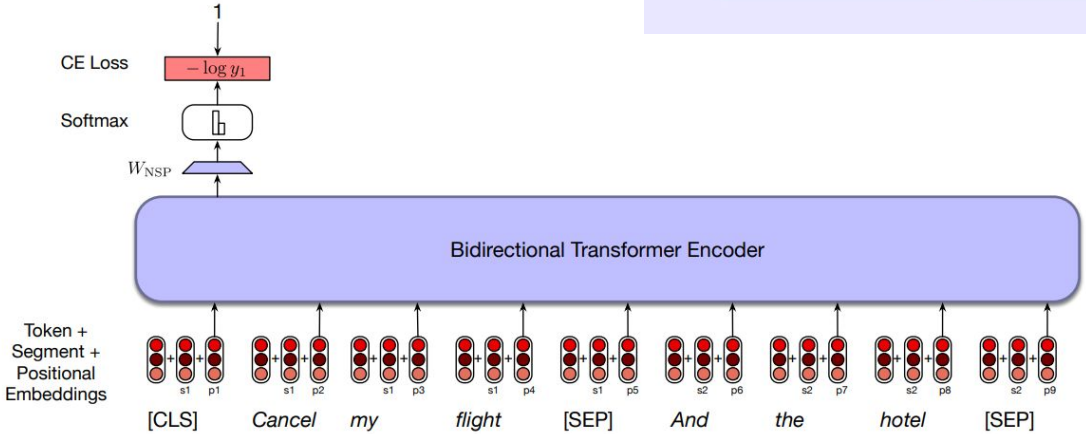


Pre-Training tasks

Next Sentence Prediction (NSP) - BERT

- Given two sentences, predict whether the second one actually follows the first one in the text.

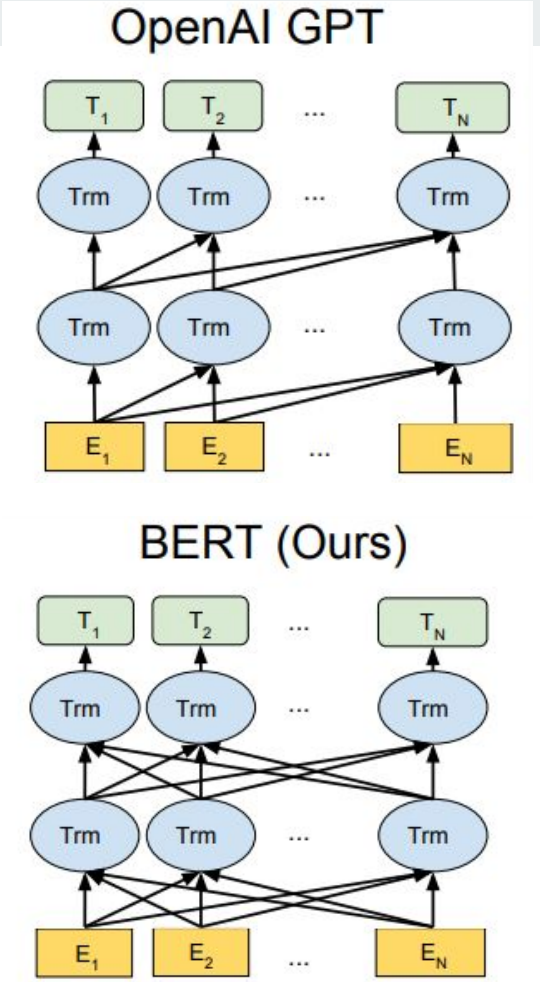
Sentence 1	Sentence 2	Next Sentence?
I have a class	I will be back by 6	✓
I have a class	Zebra is a animal	✗



During training, the output vector from the final layer associated with the [CLS] token represents the next sentence prediction. A learned set of classification weights is used to produce a two-class prediction from the raw [CLS] vector

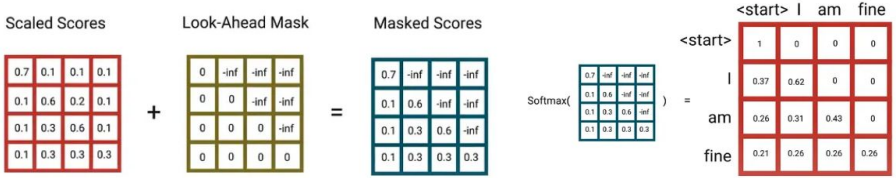
BERT vs GPT

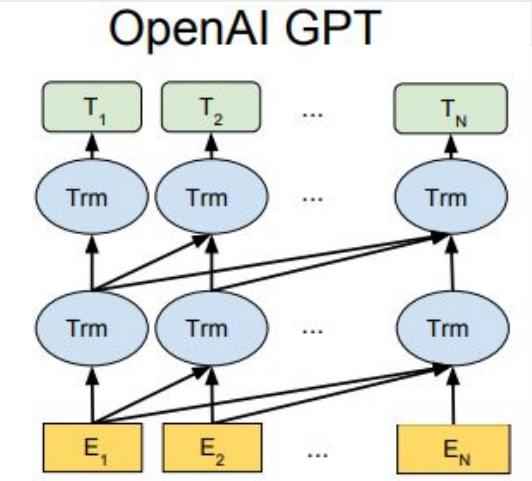
- GPT is a causal or left-to-right model
  - applied to problems such as summarization and machine translation
- Not useful in tasks such as named-entity tagging
  - require information from the right context as we process each element.
- BERT representations are jointly conditioned on both left and right context in all layers

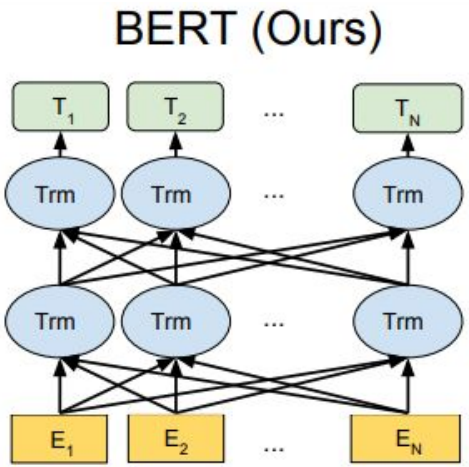


BERT vs GPT

- Mask Self-Attention
- GPT doesn't see the full input at once (unlike BERT). Instead, it only uses past tokens to predict the next.
- By seeing the “future” tokens, the model could cheat by seeing the actual token it's supposed to predict. For example, while predicting “world” in “Hello world”, it could just look ahead at “world”.



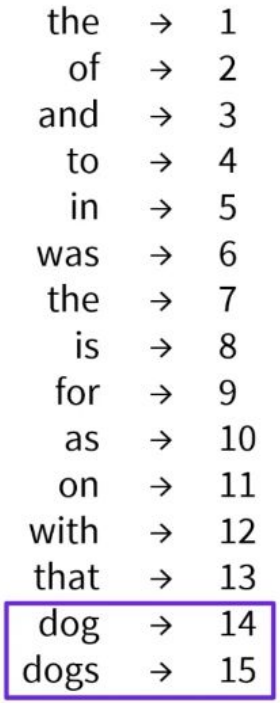




Tokenization

Word-based Tokenizer

- Very similar words have entirely different meanings
- Large vocabulary - heavy
- Unknown words
- Lost of informations



Character-based Tokenizer

Fewer word Ids
Fewer unknown words
Characters hold less information than words
Sentences generate large sequences

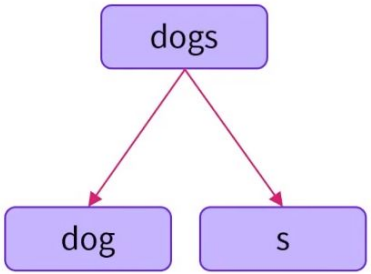
a	→	1
b	→	2
c	→	3
d	→	4
e	→	5
f	→	6
g	→	7
…	→	…
1	→	27
2	→	28
3	→	29
…	→	…
!	→	37
…		…
à	→	256

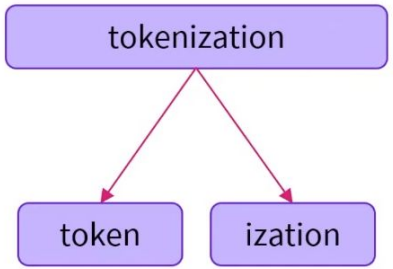
the	→	1
of	→	2
and	→	3
to	→	4
in	→	5
was	→	6
the	→	7
is	→	8
for	→	9
as	→	10
on	→	11
with	→	12
that	→	13
…		…
malapropism	→	170,000

Subword-based Tokenizer

Frequently used words should not be split into smaller subwords

Rare words should be decomposed into meaningful subwords.





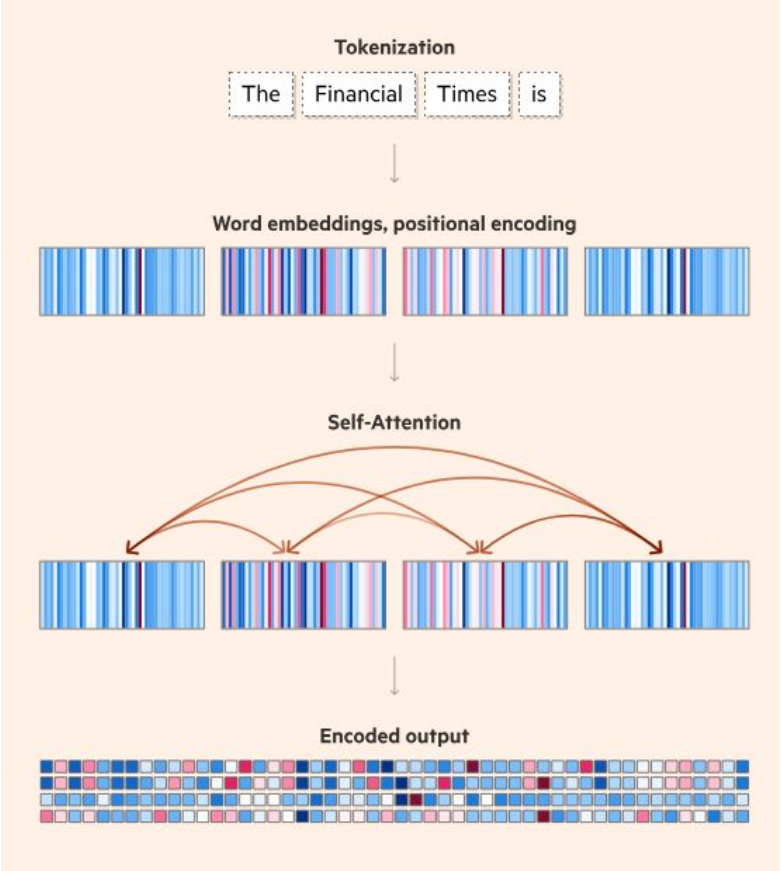
Token
Tokens
Tokenizing
Tokenization

Tokenization
Modernization
Immunization

Generating text with Decoder models

After the tokenization and encoding a user-prompt, a block of data representing the input as the machine understands it is generated,

This including meanings, positions and relationships between words.



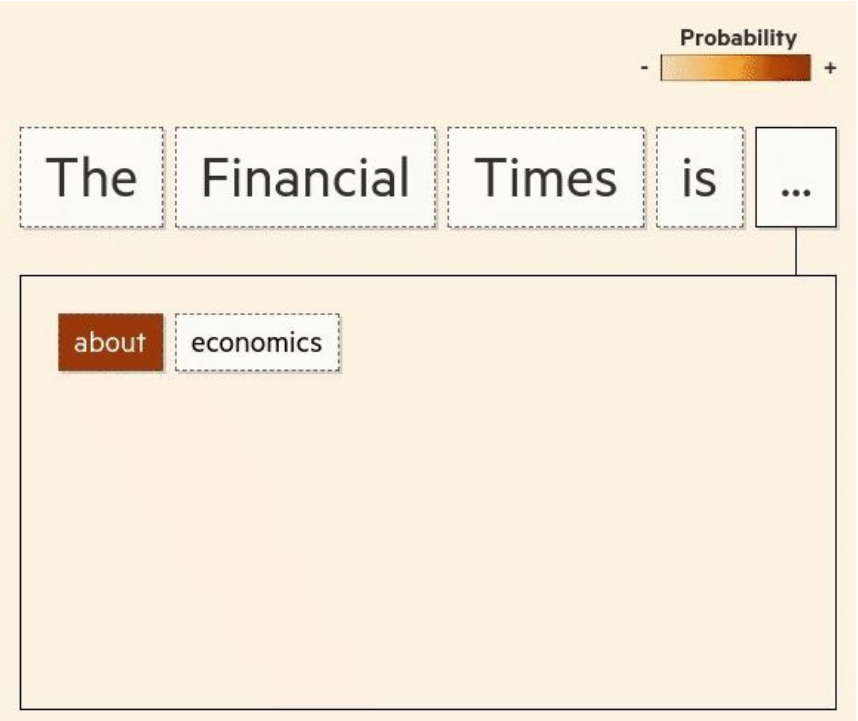
The model's aim is now to predict the next word in a sequence and do this repeatedly until the output is complete.

To do this, the model gives a probability score to each token, which represents the likelihood of it being the next word in the sequence.

And it continues to do this until it is happy with the text it has produced.

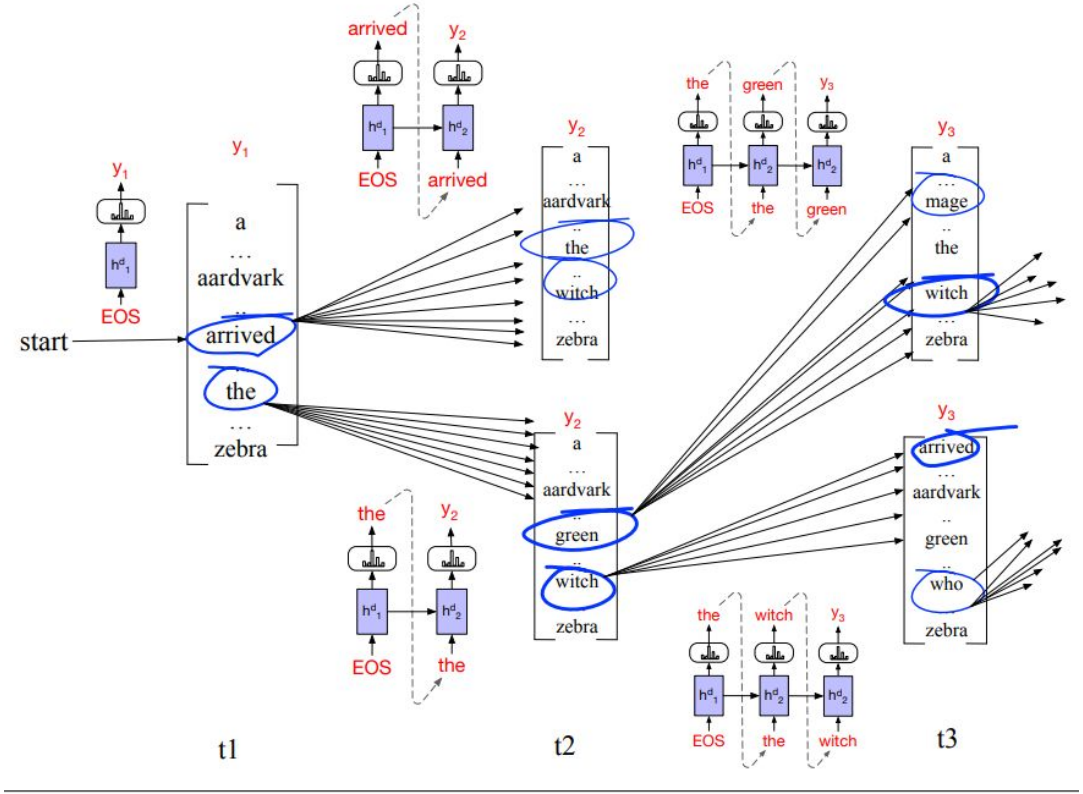
Problem: Predicting the following word in isolation (“greedy search”) can introduce problems. Sometimes, while each individual token might be the next best fit, the full phrase can be less relevant.

Solution: Rather than focusing only on the next word in a sequence, it looks at the probability of a larger set of tokens as a whole. (Beam Search)

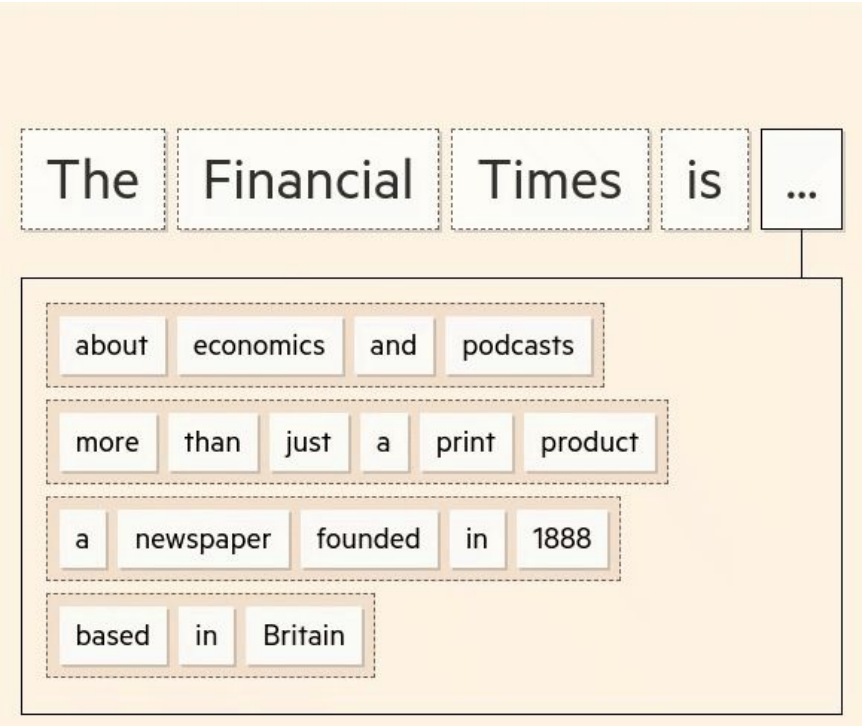


At each time step, we choose the k best hypotheses, compute the V possible extensions of each hypothesis, score the resulting k ∗V possible hypotheses and choose the best k to continue.

At time 1, the frontier is filled with the best 2 options: *arrived* and *the*. We then extend each of those, compute the probability of all the hypotheses so far (arrived the, arrived witch, the green, the witch) and compute the best 2 (in this case the green and the witch) to be the search frontier to extend on the next step.



With beam search, the model is able to consider multiple routes and find the best option, producing better and more coherent results



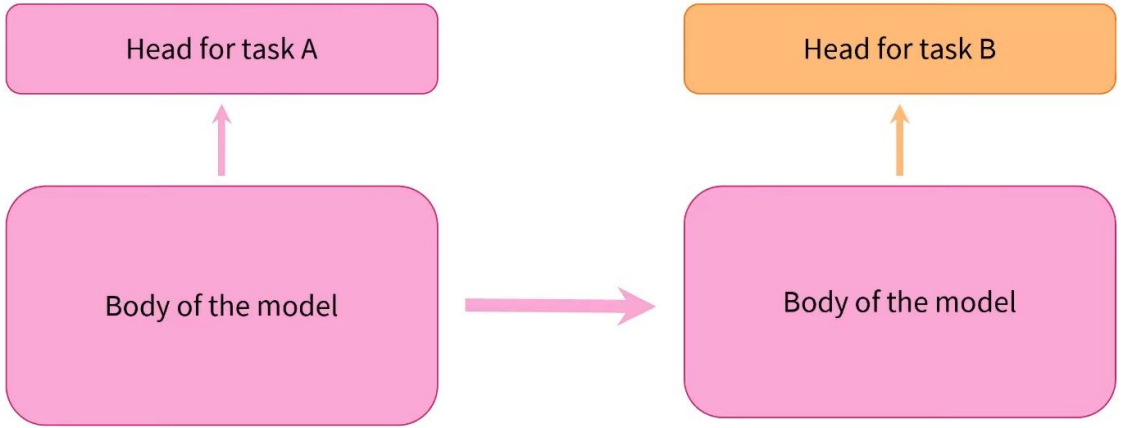
Fine-Tunning BERT

Fine-Tuning LLMS -Transfer Learning

Drop last layers that focus on the pretraining objective

Add a new head in order to fine tune the model to the new classification problem

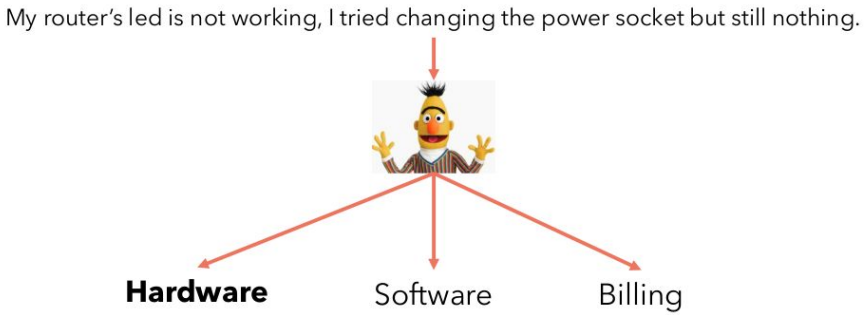
Train -> **huge dataset**
Fine-tune -> **small dataset**

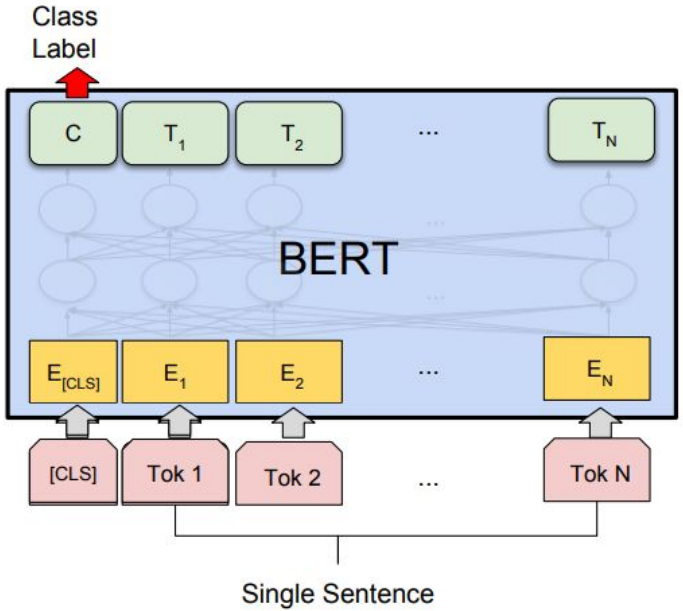


BERT - Pretrained on English wikipedia and 11k books

Text Classification

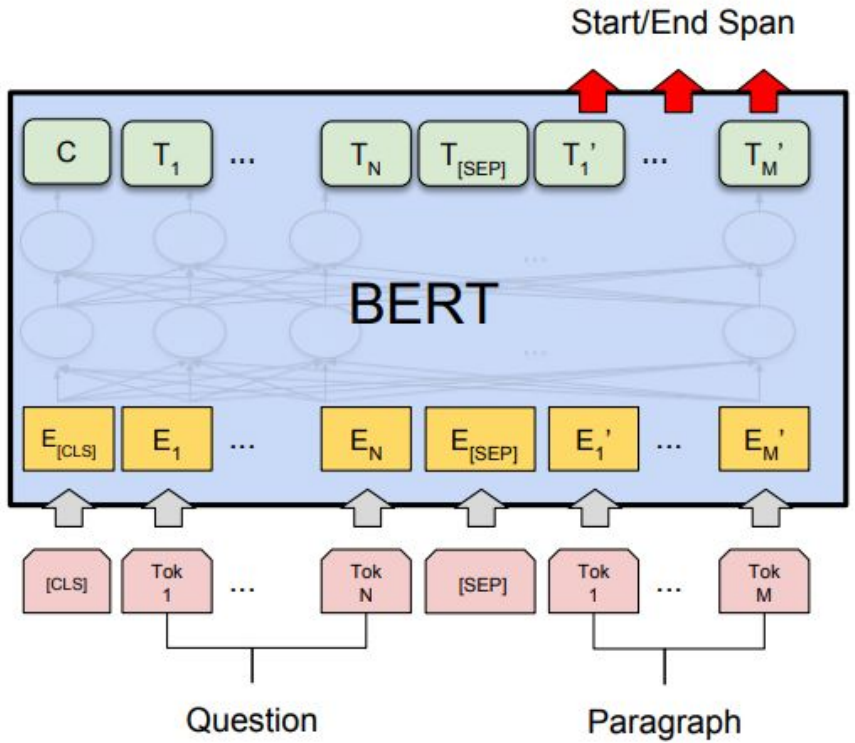
Classification tasks such as sentiment analysis are done similarly to Next Sentence classification, by adding a classification layer on top of the Transformer output for the [CLS] token.





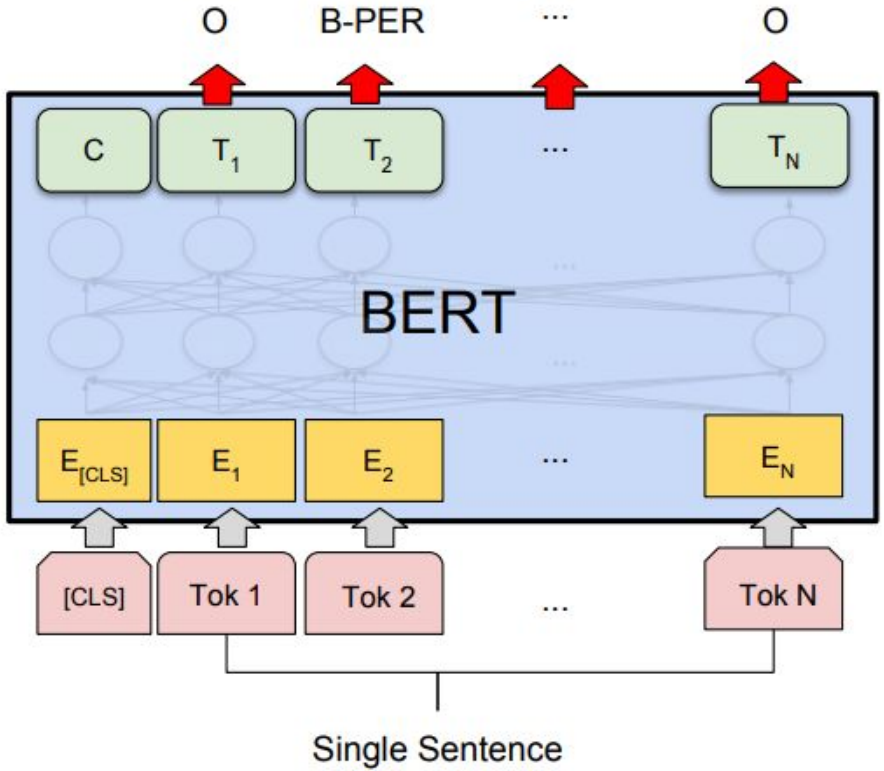
Question Answering

In **Question Answering tasks**, the software receives a question regarding a text sequence and is required to mark the answer in the sequence. Using BERT, a Q&A model can be trained by learning two extra vectors that mark the beginning and the end of the answer.



Fine-Tuning BERT - Named Entity Recognition

In **Named Entity Recognition (NER)**, the software receives a text sequence and is required to mark the various types of entities (Person, Organization, Date, etc) that appear in the text. Using BERT, a NER model can be trained by feeding the output vector of each token into a classification layer that predicts the NER label.



Transformers Library

Hugging Face

Transformers

huggingface_hub

Large Language Models
Where are we going?

Pre-training Transformers model

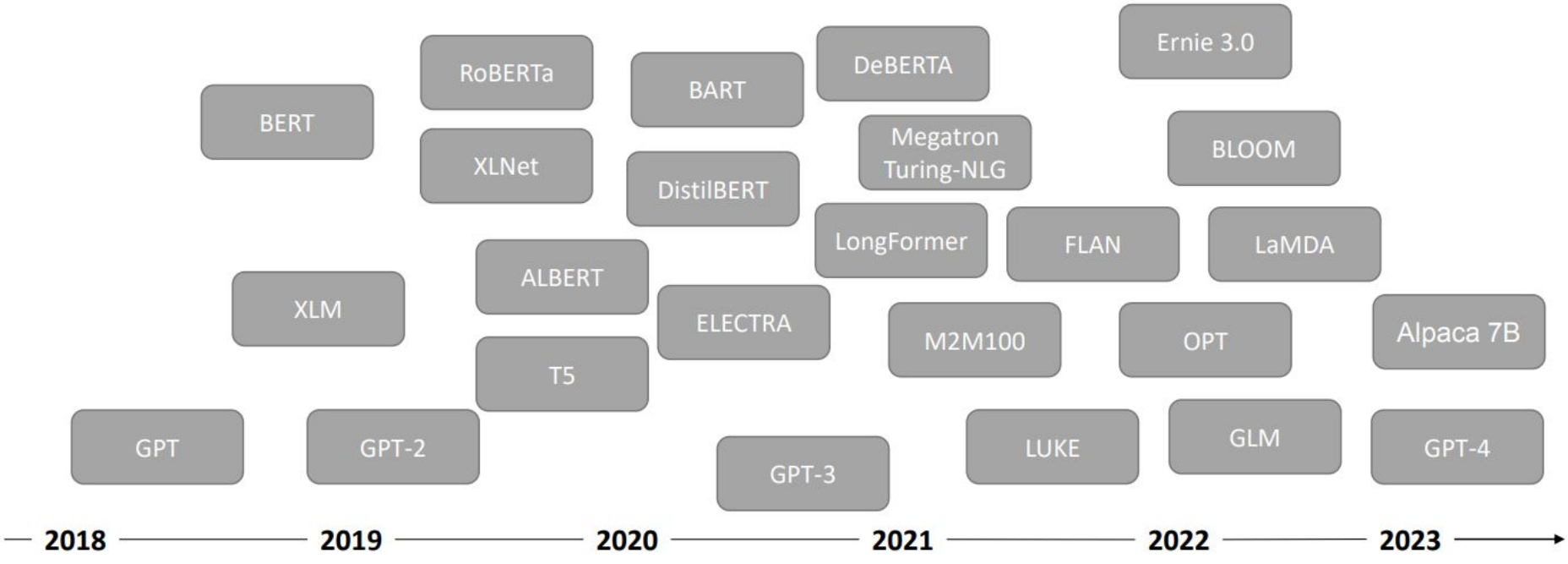
MegatronLM

Massive Transformer Language Model from NVIDIA

8.3B parameters

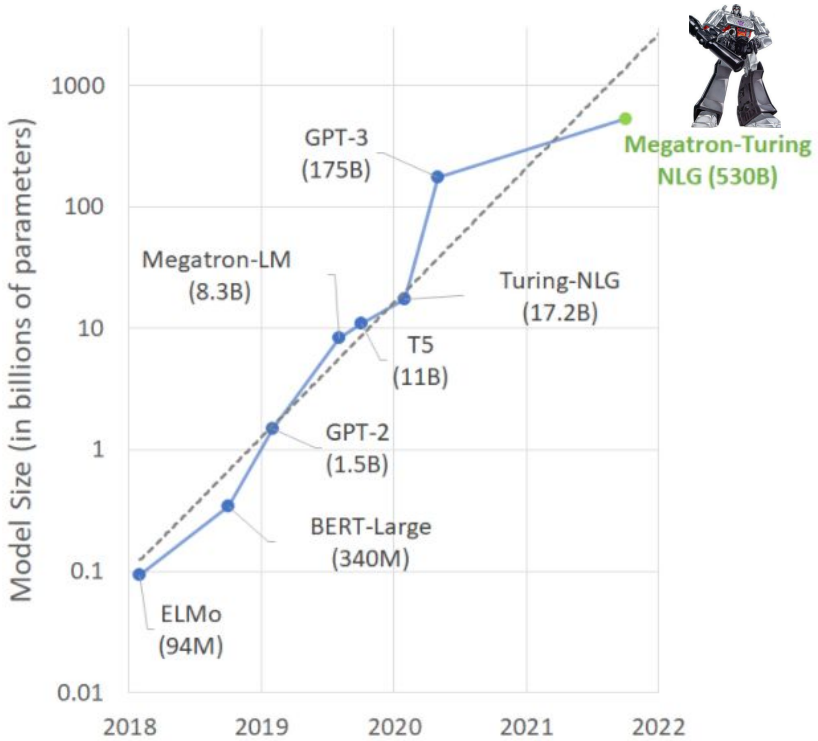
Trained on 512 GPUs for 9 days ($450k on EC2)

- **Energy concerns**


Ernie 3.0
RoBERTa
DeBERTA
BART
BERT
Megatron
Turing-NLG
BLOOM
XLNet
DistilBERT
LongFormer
FLAN
LaMDA
ALBERT
XLM
ELECTRA
M2M100
OPT
Alpaca 7B
T5
GLM
GPT
GPT-2
LUKE
GPT-4
GPT-3
2018
2019
2020
2021
2022
2023

Megatron-Turing Natural Language Generation

- Microsoft and Nvidia, October 2021
- Trained with 4480 A100 80GB GPUs
- 15 datasets consisting of a total of 339 billion tokens.



- Mistral - 7 billion parameters.
- LLaMA 3 - 70 billion parameters.
- PaLM - 540 billion parameters.
- GPT-4 is rumoured to have around 1.76 trillion parameters.
- Etc..

A larger number of parameters doesn't always guarantee better performance.

Transformers

- Context Matters

Luís Filipe Cunha
lfc@di.uminho.pt

José João Almeida
jj@di.uminho.pt